

the Euler-Maruyama Scheme in the Context of the Geometric Brownian Motion

Erich
BS Mathematics

September 20, 2026

1 Introduction

The goal of this project is two-fold: (1) to verify that the upper bound on the order of convergence of the Euler-Maruyama scheme is indeed close to the actual order of convergence, (2) to see how compatible it is with GPU computing.

2 Preliminaries

2.1 Introduction to SDEs

A stochastic differential equation (SDEs) is a differential equation of this specific form:[1].

$$\begin{aligned}dX_t &= b(X_t, t)dt + \sigma(X_t, t)dB_t \\ X_0 &= x, \text{ where } x \text{ is a fixed real number.}\end{aligned}\tag{1}$$

There are restrictions on b, σ in order for the scheme to converge; specifically, we need Lipschitz continuity on the x -variable. In any case, the only important thing to note is that the geometric brownian motion satisfies that condition.

For a bit more context, $B_t \sim \text{Normal}(\mu = 0, \sigma^2 = t)$. Another important thing to note is that increments of such random variables are independent as long as the time intervals they are looking at are disjoint. This will become central when creating the actual kernel.

2.2 the Geometric Brownian Motion

The geometric brownian motion is an SDE with the specific form: [1].

$$\begin{aligned}dX_t &= bX_tdt + \sigma X_tdB_t \\ X_0 &= x, \text{ where } x \text{ is a fixed real number.}\end{aligned}\tag{2}$$

This SDE is of the rare type that has a closed form solution, which is needed in order to compare a numerical solution to the theoretical correct solution. The derivation for the solution is not relevant and would need the explanation of Ito's lemma to show and thus is omitted. The closed form solution is simply stated instead:

$$X_t = xe^{(b-\frac{\sigma^2}{2})t+\sigma B_t} \quad (3)$$

2.3 the Euler-Maruyama Scheme and Its Order of Convergence

The Euler-Maruyama scheme approximates equation (1) at the specified discrete times $0 = t_0 < t_1 < t_2 < \dots < t_{k-1} < t_k = T$. The scheme is specifically the following:[1].

$$\hat{X}_{t_k} = \hat{X}_{t_{k-1}} + b(\hat{X}_{t_{k-1}}, t_{k-1})\Delta t_{k-1} + \sigma(\hat{X}_{t_{k-1}}, t_{k-1})\Delta B_{k-1} \quad (4)$$

To obtain values outside of these specified times, the values are interpolated to give an approximation. Theoretically, the actual random variables are interpolated, but in practice, the evaluated values are the ones being interpolated. The difference between these techniques is simply that the Brownian Motions are interpolated for the inbetween values instead of being randomly generated.

Furthermore, the scheme has the following order of convergence formula in the case that the discrete times are equally spaced, with common length Δt :

$$E[\sup_{0 \leq t \leq T} |\hat{X}_t - X_t|^p] \leq C\Delta t^{p/2}. \quad (5)$$

This is the object, that is under consideration. This project aims to verify this for the case that $p = 2$ to see whether this upper bound is indeed a sharp enough upper bound for this numerical scheme.

2.4 Numerical Method for Expectation

The strong law of large numbers will be used to approximate the expected value with the sample mean: Suppose the number of trials we performed was N , [1].

$$E[\sup_{0 \leq t \leq T} |\hat{X}_t - X_t|^2] \approx \frac{\sum \sup |\hat{X}_t - X_t|^2}{N} \quad (6)$$

Each supremum in this case is 1 trial run.

3 Practical Considerations

Before moving on to the programming portion, it helps to demystify the SDE equation and put some actual variables in to aid the reader.

For simplicity, consider the solution of the SDE from $t \in [0, 1]$. This means that the output of the SDE is a sequence of random variables that gives us the random trajectory of some object subject to that stochastic differential equation from 0 to 1.

Choose equally-spaced time steps with equal lengths $h = \frac{1}{n}$, where $n \in \mathbb{N}$ is the number of time steps desired. Thus, the scheme will give approximations at times $0, \frac{1}{n}, \frac{2}{n}, \frac{3}{n}, \dots, 1$. Translating this into the notation in equation (4):

$$\Delta t_{k-1} = h \text{ and } \Delta B_{k-1} \sim \text{Normal}(0, h) \quad (7)$$

Lastly, plugging in the specific SDE under consideration, we obtain the following scheme:

$$\hat{X}_{kh} = \hat{X}_{(k-1)h} + b \cdot \hat{X}_{(k-1)h} \cdot h + \sigma \cdot \hat{X}_{(k-1)h} \cdot (\sqrt{h} \cdot Z) \quad (8)$$

, where Z is a standard normal random variable.

4 Implementation

4.1 Device Properties

For context, I am using an RTX3070 and will be using CUDA C. Here are the relevant details of my specific device:

Table 1: Device Properties	
GPU Property	Value
GPU Clock Rate	1.725 GHz
Theoretical Memory Bandwidth*	441.6 GB/s
Registers per SM	65,536
Max Threads per SM	1,536

* this is calculated from $\text{MemoryClockRate} \times \text{BusWidth} \times \text{DataRateMultiplier} \div 10^9$ [2]

4.2 Technical Issues

It is worth mentioning the sheer difficulty of making curanddx work. It is not allowed to use CUDA Runtime API like `kernel<<<>>>()`. Curanddx forces its users into low-level CUDA Driver API. Just some of the code needed to make all this work is in Appendix A.2. This code is motivated by mathdx documentation.[4]

There were also several other problems like some of the CUDA directories actually being moved around in the newer versions but the official CUDA NVRTC sample code still assumes the older directory.

Lastly, they cannot be mix-and-matched as CUDA Runtime API creates its own context different from any context one would have created using NVRTC code. Suddenly using `cudaMalloc()` while already using NVRTC code like `CuMemAlloc()` will create a new context and mess up the entire program.

4.3 Estimating the Order of Convergence

4.3.1 Calculating the Supremum Term...or Not

If we consider instead $|\hat{X}_1 - X_1|^2$, we avoid all the supremum computations, at the cost of strictly worse order of convergence. As will be seen though, even this strictly worse method for calculating the order of convergence still gives us a number close to $\frac{p}{2}$.

In any case, I did also create a kernel for the original supremum term; the code is not shown otherwise the entire pdf would just be code blocks.

The generation of brownian motion is handled by `curanddx`. [6]

Here is the actual CUDA kernel for the non-supremum kernel:

```

1      // ... Boilerplate initialization and cuRANDdx setup omitted
      ↪ ...
2
3
4      double S = x;
5      const double drift = b * h;
6      const double std_dev = sqrt(h); // Standard deviation is
      ↪ sqrt(dt)
7
8      // Euler-Maruyama Scheme
9      for (int iter = 0; iter < iterations; iter++) {
10         // Generate standard normal and scale by sqrt(h)
11         double dB = dist.generate(rng) * std_dev;
12         B_total += dB;
13
14         // S = S + S * (b*dt + s*dW)
15         // Using fma for double precision (fmaf is for float)
16         double change_factor = fma(s, dB, drift);
17         S = fma(S, change_factor, S);
18     }
19     // generate closed form output
20     // (b - 0.5 * s * s) * T + s * B_total
21     double exponent = fma(s, B_total, (b - 0.5 * s * s) * T);
22     double closed_form = x * exp(exponent);
23
24     squared_error[tid] = (S - closed_form) * (S - closed_form);
25 }

```

Notes:

1. Why curanddx over the legacy curand_kernel.h?

Curanddx RNGs have a massive advantage over curand_kernel.h RNGs in that it stores the states of the RNG inside the registers whereas the legacy method was to constantly read and write the states needed to use the RNG from global memory. [5]

2. RNG considerations

Notice that the B term used in the closed form version is accumulated from the generated ΔB_{k-1} generations. This was not done to improve performance. On the contrary, it actually increases register usage per thread. This is necessary because the same generated term must be used for the same sample path.

3. Performance Improvements on the actual computation

Note that in equation (7), we can factor out $\hat{X}_{(k-1)h}$ to get

$$\hat{X}_{kh} = \hat{X}_{(k-1)h}(1 + b \cdot h + \sigma\sqrt{h} \cdot Z) \quad (9)$$

Since $b \cdot h$ is constant, we calculate it at the start and stored it in the variable called *drift*. FMA (fused-multiply-add) was used to increase GFLOPs (one fma is one clock cycle instead of the two it will usually need).

4. Why *double* instead of *float*?

The extra precision here is absolutely crucial as the squared error terms per sample are very small.

4.3.2 Calculating the Expected Value

Parallel prefix sums was used on the sum in the RHS of equation (6). There are two versions of the parallel prefix sum used. A naive kernel was also used just for benchmarking; the code is omitted though.

Here is the first version, obtained from lecture (6), but is cited from Kirk&Hwu, copying the citation in the lecture slide. [3]

```
1  __global__ void BlockParallelScan(double* Input, double* Output,
   ↪ double* Intermediate, int InputSize) {
2      __shared__ double XY[SECTION_SIZE];
3
4      int i = blockIdx.x * blockDim.x + threadIdx.x;
5      if (i < InputSize) { XY[threadIdx.x] = Input[i]; }
6      else { XY[threadIdx.x] = 0.0f; }
7
8      for (unsigned int stride = 1; stride < SECTION_SIZE;
   ↪ stride *= 2) {
9          __syncthreads();
```

```

10
11         int index = (threadIdx.x + 1) * 2 * stride - 1;
12         if (index < SECTION_SIZE) XY[index] += XY[index -
13             ↳ stride];
14     }
15
16     for (int stride = (SECTION_SIZE) / 4; stride > 0; stride
17         ↳ /= 2) {
18         __syncthreads();
19         int index = (threadIdx.x + 1) * stride * 2 - 1;
20         if (index + stride < SECTION_SIZE) XY[index +
21             ↳ stride] += XY[index];
22     }
23     __syncthreads();
24
25     if (i < InputSize) { Output[i] = XY[threadIdx.x]; }
26     if (threadIdx.x == SECTION_SIZE - 1 && i < InputSize) {
27         ↳ Intermediate[blockIdx.x] = XY[threadIdx.x]; }
28 }
29
30 __global__ void IntermediateParallelScan(double* Intermediate,
31     ↳ int IntermediateSize) {
32     __shared__ double XY[SECTION_SIZE];
33
34     int i = blockIdx.x * blockDim.x + threadIdx.x;
35     if (i < IntermediateSize) { XY[threadIdx.x] =
36         ↳ Intermediate[i]; }
37     else { XY[threadIdx.x] = 0.0f; }
38
39     // ... same Parallel Reduction logic as in
40     ↳ ParallelBlockScan and is omittedd ...
41     __syncthreads();
42
43     if (i < IntermediateSize) { Intermediate[i] =
44         ↳ XY[threadIdx.x];}
45 }
46
47 __global__ void AddBack(double* Output, double* Intermediate, int
48     ↳ InputSize) {
49     int i = (blockIdx.x + 1) * blockDim.x + threadIdx.x;
50
51     if (i < InputSize) {
52         Output[i] += Intermediate[blockIdx.x];
53     }
54 }

```

47 }

All this is necessary because the sample sizes are in the hundreds of millions and so the intermediate add-back step is necessary; however, noting that only the total sum is needed. Thus, an efficient kernel should skip calculating most of the partial sums.

That is the motivation for this second kernel:

```
1  __global__ void ReductionKernel(const double* input, double*
   ↪ output, int n) {
2      extern __shared__ double XY[];
3
4      unsigned int tid = threadIdx.x;
5      unsigned int i = blockIdx.x * blockDim.x + tid;
6
7      if (i < n) XY[tid] = input[i];
8      else XY[tid] = 0.0f;
9
10     __syncthreads();
11
12
13     for (unsigned int s = blockDim.x / 2; s > 0; s /= 2 ) {
14         if (tid < s) {
15             XY[tid] += XY[tid + s];
16         }
17         __syncthreads();
18     }
19
20     if (tid == 0) {
21         output[blockIdx.x] = XY[0];
22     }
23
24
25 }
```

This kernel needs to be paired with a while loop to handle cases where we need multiple kernel launches, due to how large the array to be summed is. A typical while loop that uses this kernel is the following:

```
1  while (n_current > 1) {
2      int gridSize = (n_current + threads_per_block - 1) /
   ↪ (threads_per_block);
3
4      if (gridSize == 1) {
```

```

5      ReductionKernel << <1, threads_per_block, shared_mem_size
        ↪ >> > (temp_input, final_time, n_current);
6      n_current = 1;
7  }
8  else {
9      ReductionKernel << <gridSize, threads_per_block,
        ↪ shared_mem_size >> > (temp_input, temp_output,
        ↪ n_current);
10
11      double* prev_input = temp_input;
12      temp_input = temp_output;
13      temp_output = prev_input;
14      n_current = gridSize;
15  }
16 }

```

5 Analysis of Kernels and Justification of Launch Configurations

5.1 GBM Kernels

Here are some essential facts about the kernel. Since there are no memory-related concerns for this kernel, it is primarily computation-bounded and as such the relevant metrics are occupancy and achieved GFLOPs. The relevant data is presented below. (total FLOPs was obtained by manually counting the number of operations in the kernel)

Table 2: GBM Kernel Properties

Property	Value
Register Usage per Thread	37
Total Floating Point Operations per Thread (non-supremum)*	$6n + 10$
Total Floating Point Operations per Thread (supremum)*	$15n + 6$

*Note that counting the total floating point operations per thread does not count the crucial amount of GFLOPs for the actual RNG since it is not something I programmed. We can still calculate meaningful GFLOPs from the numbers given in the table though, as low achieved GFLOPs would mean a significant portion of the time is spent just on generating the normal distributions.

The launch configurations for this kernel is basic, maximize occupancy. Note

that

$$\begin{aligned}
(\text{registers per thread}) \times (\text{max threads per SM}) &= (37) \times (1536) \\
&= 61,440 \\
&< 65,536 \\
&(\text{registers per SM})
\end{aligned} \tag{10}$$

Therefore, register usage will not be a concern. Thus, there are 256 threads per block and 6 blocks per SM.

5.2 Parallel Sum Kernels

Since the parallel prefix sum kernels are memory-bound, it should be useful to compare effective with theoretical bandwidth. Just to note, N here refers to the number of samples and M here refers to the number of blocks, which is $\frac{N}{\text{threads per block}}$.

We have the following results (obtained by manually counting the number of reads and writes):

$$\text{Total Bytes Transferred (naive-kernel): } (8 \text{ bytes [double]}) 4N + 4M \tag{11}$$

$$\text{Total Bytes Transferred (improved-kernel): } N + \frac{2N}{\text{threads per block}} + \frac{2 \frac{N}{\text{threads per block}}}{\text{threads per block}} + \dots \tag{12}$$

For the naive parallel sum, the aim was to maximize the number of threads per block, which is 1024, even at the expense of occupancy because the shared memory tiling is per block and also the reduction means the terms that are partially summed together after the *BlockParallelScan* kernel is ran are per block. So even if 256×6 launch configurations would maximize occupancy, it would mean the sums would only be grouped 256 terms at a time. Lastly, a 256 reduction at a time would actually require more than 1 run of the logic for larger sample sizes, which the shown code cannot handle.

For the improved parallel sum, the while loop can handle any sample size, and thus we are happy to use 256×6 launch configurations.

6 Data Analysis

6.1 log-log Linear Regression for Order of Convergence

To reveal the $\frac{p}{2}$ exponent term in equation (5), take the logarithm of both sides. This means that approximately:

$$\frac{p}{2} \log(\Delta t) \approx \text{expected value of squared error} \tag{13}$$

Thus, doing a log-log regression, it should be expected that the slope of the regression line is approximately $\frac{p}{2} = 1$ for the $p = 2$ case.

It is quite surprising how sharp the bound is because the proof of the bound uses quite crude inequalities. The confusing part is that the supremum order of convergence is worse at only 0.87. I am unable to explain why.

The data comes from using sample size $= N = 100,000,000$ with step sizes $n = 16, 32, 48, 64, 80, 96, 112, 128, 144, 160$.

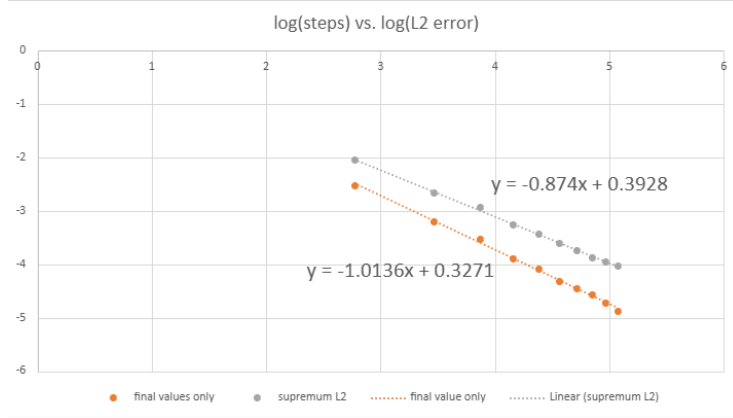


Figure 1: $\log(\text{steps})$ vs. $\log(\text{squared error})$

6.2 Analysis of GBM Kernel

For $N = 100,000,000$ and iterations $n = 160$, the mean runtime of the GBM kernel is 6363.420947(ms).

Plugging this into the total floating point operations per thread, multiplying the number of samples, and then dividing this by our runtime in seconds, we actually get

$$N \frac{6n + 10}{\text{runtime (in sec)}} = 15.24 \text{ GFLOPs.} \quad (14)$$

This is only about 5% of the theoretical (double) GFLOPs of the RTX3070 which is 317.4 GFLOPs. This is not that surprising though as, again, we did not include the operations needed to generate the random numbers, which probably takes up a significant amount of the computation time, and also wrote off the exponential operator as 1 operation.

Doing the same thing for the supremum GBM kernel, the achieved GFLOPs is 28.2 GFLOPs, which means we attain 10% of the theoretical (double) GFLOPs. It makes sense that it is more efficient since it performs more basic operations per loop without using more registers.

6.3 Analysis of the Parallel Sum Kernels

For $N = 100,000,000$ and iterations $n = 160$ with our specified launch configurations from section 5, we plug in the numbers and get the following effective bandwidths:

Table 3: Effective Bandwidth

Kernel	runtime (ms)	Effective Bandwidth	Efficiency*
Naive	50.4379681 ms	63.51 GB/s	14.3%
Improved	13.9439136 ms	57.82 GB/s	13.1%

*Efficiency means relative compared to the theoretical maximum bandwidth we presented in section 4.

But wait, the effective bandwidth is roughly the same. This is expected since those two kernels roughly have the same logic and need to sum the same amount of numbers. The non-naive kernel is still better though since it has a lower runtime. This is because the improved kernel needs to pass strictly less data around.

7 Conclusions and Recommendations

7.1 Unexplained 0.87 Rate of Convergence

A future project could do error analysis to look into why exactly doing a supremum over errors creates a vastly lower rate of convergence.

7.2 Occupancy Issues

The most important result in all of this is that the GBM kernel, even for 1-Dimensional SDEs, already maximizes register usage ($\sim 61,000$ vs. $\sim 64,000$). Higher-dimensional SDEs will undoubtedly make use of more registers, preventing maximal occupancy from being achieved.

Furthermore, even for 1-Dimensional SDEs, most SDEs are not as clean as the GBM, needing more variables and having more arithmetic performed just to evaluate the drift and diffusion functions at a point.

Using higher order of convergence numerical schemes will run into even more occupancy problems as typically the improvement in order of convergence between schemes multiplies the number of operations and variables needed by the same amount.

7.3 Calculating the Expected Value of the Squared Errors was Not the Limiting Factor

Coming into this project, I planned the improved parallel summation because I thought summing hundreds of millions of terms was going to be the bottleneck.

In part (B) of the appendix, you can see the vast difference in runtimes between the GBM kernel and the parallel reduction kernel(s).

In future projects related to SDE schemes, there is no need to be worried of calculating expectations.

7.4 Future Directions

It would be instructive to try and compare runtimes and efficiencies with FEM schemes for Diffusion SDEs. Interested readers are advised to look into the following:

Appendices

Appendix A: Important Codes

Appendix B: Hardware Configuration & Execution Logs

A :Complete Codes

A.1 Complete Supremum GBM Kernel

```
1  const std::string gbm_sup = R"kernel(  
2  #include <curanddx.hpp>  
3  
4  using RNG = decltype(curanddx::Generator<curanddx::pcg>() +  
    ↪ curanddx::SM<RNG_SM>() + curanddx::Thread());  
5  
6  extern "C" __global__ void gbmKernel_sup(  
7      double* squared_error,  
8      const double x,  
9      const double b,  
10     const double s,  
11     const int iterations,    // n  
12     const double T,  
13     const unsigned long long seed,  
14     const typename RNG::offset_type offset  
15 )  
16 {  
17     const int tid = blockDim.x * blockIdx.x + threadIdx.x;  
18     double h = T / iterations;  
19     double B_total = 0;  
20  
21     RNG rng(seed, ((offset + tid) % 65536), ((offset + tid) /  
    ↪ 65536));  
22  
23     double S = x;  
24     const double drift = b * h;  
25     const double std_dev = sqrt(h); // Standard deviation is  
    ↪ sqrt(dt)  
26     const double b_minus_pt5s_squared = b - 0.5 * s * s;  
27  
28     // Using the same generator object for efficiency  
29     curanddx::normal<double, curanddx::icdf> dist(0.0, 1.0);  
30  
31     double max_squared_error = 0;  
32     double elapsed_h = 0;  
33  
34     for (int iter = 0; iter < iterations; iter++) {  
35         elapsed_h += h;  
36  
37         // Generate standard normal and scale by sqrt(h)  
38         double dB = dist.generate(rng) * std_dev;  
39         B_total += dB;
```

```

40
41 // S = S + S * (b*dt + s*dW)
42 // Using fma for double precision (fmaf is for float)
43 double change_factor = fma(s, dB, drift);
44 S = fma(S, change_factor, S);
45
46 // generate closed form output
47 // (b - 0.5 * s * s) * T + s * B_total
48 double exponent = fma(s, B_total,
49 ↪ (b_minus_pt5s_squared)*elapsed_h);
50 double closed_form = x * exp(exponent);
51
52 double temp_squared_error = (S - closed_form) * (S -
53 ↪ closed_form);
54
55 max_squared_error = fmax(max_squared_error,
56 ↪ temp_squared_error);
57 }
58 squared_error[tid] = max_squared_error;
59 }
60 )kernel";

```

A.2 NVRTC Object Code To Simplify Kernel Launches

```

1 class GpuContext {
2 public:
3     CUdevice    cuDevice;
4     CUcontext    context;
5     GpuContext() {
6         int current_device;
7         CU_CHECK_AND_EXIT(cudaGetDevice(&current_device));
8
9         CU_CHECK_AND_EXIT(cuInit(0));
10        CU_CHECK_AND_EXIT(cuDeviceGet(&cuDevice,
11 ↪ current_device));
12        #if CUDA_VERSION >= 13000
13        CU_CHECK_AND_EXIT(cuCtxCreate(&context,
14 ↪ (CUctxCreateParams*)0, 0, cuDevice));
15        #else
16        CU_CHECK_AND_EXIT(cuCtxCreate(&context, 0, cuDevice));
17        #endif
18    }
19    ~GpuContext() { cuCtxDestroy(context); }
20 };

```

```

19
20 class GpuKernel {
21 private:
22     CUmodule    module;
23     CUfunction kernel;
24
25 public:
26     GpuKernel(GpuContext& ctx, const std::string& source, const
        ↪ std::string& name) {
27
28         int current_device;
29         CU_CHECK_AND_EXIT(cudaGetDevice(&current_device));
30
31         // Create NVRTC program
32         std::string kernel_name_cu = name + ".cu";
33         nVRTCProgram program;
34         NVRTC_SAFE_CALL(nVRTCCreateProgram(&program,           //
        ↪ program
35             source.c_str(),           // buffer
36             kernel_name_cu.c_str(), // name
37             0,                        // numHeaders
38             NULL,                     // headers
39             NULL));                   // includeNames
40
41         // Prepare compilation options
42         std::vector<const char*> opts = {
43             "--std=c++17",
44             "--device-as-default-execution-space",
45             "-I" CUDA_INCLUDE_DIR, // Add path to CUDA include
        ↪ directory
46             "-I" CUDA_CCCL_INCLUDE_DIR
47         };
48
49         // This function should theoretically be able to handle
        ↪ all the directories without having to
50         // manually type the directories as I've done above, but
        ↪ sadly get_curanddx_include_dirs actually cannot find
        ↪ CCCL.
51         std::vector<std::string> curanddx_include_dirs =
        ↪ nVRTC::get_curanddx_include_dirs();
52         for (auto& d : curanddx_include_dirs) {
53             opts.push_back(d.c_str());
54         }
55
56         // Add GPU_ARCHITECTURE definition to opts
57         std::string gpu_architecture_definition =

```

```

58         "-DRNG_SM=" +
        ↪ std::to_string(nvrtc::get_device_architecture(current_device)
        ↪ * 10);
59 opts.push_back(gpu_architecture_definition.c_str());
60
61 // Add gpu-architecture to opts
62 std::string gpu_architecture_option =
        ↪ nvrtc::get_device_architecture_option(current_device);
63 opts.push_back(gpu_architecture_option.c_str());
64
65 // Compile the kernel
66 nvrtcResult compileResult = nvrtcCompileProgram(program,
        ↪ // program
67         static_cast<int>(opts.size()), // numOptions
68         opts.data()); // options
69
70 // Obtain compilation log from the program
71 if (compileResult != NVRTC_SUCCESS) {
72     for (auto o : opts) {
73         std::cout << o << std::endl;
74     }
75     nvrtc::print_program_log(program);
76     std::exit(1);
77 }
78
79 // Obtain cubin from the program.
80 size_t cubin_size;
81 NVRTC_SAFE_CALL(nvrtcGetCUBINSize(program, &cubin_size));
82 auto cubin = std::make_unique<char[]>(cubin_size);
83 NVRTC_SAFE_CALL(nvrtcGetCUBIN(program, cubin.get()));
84
85 // Destroy the program.
86 NVRTC_SAFE_CALL(nvrtcDestroyProgram(&program));
87
88 // Set the passed context as "current" for this thread
89 cuCtxPushCurrent(ctx.context);
90
91 CU_CHECK_AND_EXIT(cuModuleLoadDataEx(&module, cubin.get(),
        ↪ 0, 0, 0));
92 CU_CHECK_AND_EXIT(cuModuleGetFunction(&kernel, module,
        ↪ name.c_str()));
93
94 // get register usage of kernel function [uncomment to
        ↪ check the kernel register usage]
95 //int regs = 0;

```

```

96         //cuFuncGetAttribute(&regs, CU_FUNC_ATTRIBUTE_NUM_REGS,
           ↪ kernel);
97
98         //std::cout << "Kernel: " << name << " | Registers: " <<
           ↪ regs << std::endl;
99
100         CUcontext dummy;
101         cuCtxPopCurrent(&dummy);
102     }
103
104     ~GpuKernel() { cuModuleUnload(module); }
105
106     template <typename... Args>
107     void launch(unsigned gx, unsigned gy, unsigned gz,
108                unsigned bx, unsigned by, unsigned bz, Args... args) {
109         void* params[] = { &args... };
110         cuLaunchKernel(kernel, gx, gy, gz, bx, by, bz, 0, 0,
           ↪ params, 0);
111         cuCtxSynchronize();
112     }
113
114     template <typename... Args>
115     float launch_timed(const std::string& label,
116                      unsigned gx, unsigned gy, unsigned gz,
117                      unsigned bx, unsigned by, unsigned bz,
           ↪ Args... args) {
118
119         CUevent start, stop;
120         cuEventCreate(&start, CU_EVENT_DEFAULT);
121         cuEventCreate(&stop, CU_EVENT_DEFAULT);
122
123         // Record the start event
124         cuEventRecord(start, 0);
125
126         // Perform the actual launch
127         launch(gx, gy, gz, bx, by, bz, args...);
128
129         // Record the stop event
130         cuEventRecord(stop, 0);
131         cuEventSynchronize(stop);
132
133         float milliseconds = 0;
134         cuEventElapsedTime(&milliseconds, start, stop);
135
136         std::cout << "[GPU Kernel: " << label << "] Time: " <<
           ↪ milliseconds << " ms" << std::endl;

```

```

137         cuEventDestroy(start);
138         cuEventDestroy(stop);
139
140         return milliseconds;
141     }
142
143     template <typename... Args>
144     float launch_timed(unsigned gx, unsigned gy, unsigned gz,
145         unsigned bx, unsigned by, unsigned bz, Args... args) {
146
147         CUevent start, stop;
148         cuEventCreate(&start, CU_EVENT_DEFAULT);
149         cuEventCreate(&stop, CU_EVENT_DEFAULT);
150
151         // Record the start event
152         cuEventRecord(start, 0);
153
154         // Perform the actual launch
155         launch(gx, gy, gz, bx, by, bz, args...);
156
157         // Record the stop event
158         cuEventRecord(stop, 0);
159         cuEventSynchronize(stop);
160
161         float milliseconds = 0;
162         cuEventElapsedTime(&milliseconds, start, stop);
163         cuEventDestroy(start);
164         cuEventDestroy(stop);
165
166         return milliseconds;
167     }
168 };
169

```

Appendix B: Raw Execution Logs (100M Samples, 160 Iterations)

Appendix B: Complete Raw Execution Timing Matrix

This appendix collects the complete, unreduced execution logs.

Table 4: Raw Runtimes: Non-Supremum Simulation + Naive Parallel Scan (ms)

Trial Index	GBM Kernel (<i>gbmKernel</i>)	Naive Parallel Scan (<i>BlockParallelScan</i>)
Trial 1	6392.230957	50.365446
Trial 2	6357.598633	50.354015
Trial 3	6356.777344	50.367199
Trial 4	6357.382324	50.840576
Trial 5	6356.950195	50.360737
Trial 6	6356.540527	50.344353
Trial 7	6357.352539	50.374207
Trial 8	6383.270020	50.359295
Trial 9	6357.572266	50.336769
Trial 10	6358.534668	50.366047
Mean (μ)	6363.420947	50.406864

Table 5: Raw Runtimes: Supremum Simulation + Naive Parallel Scan (ms)

Trial Index	Supremum GBM Kernel (<i>gbmKernel_sup</i>)	Naive Parallel Scan (<i>BlockParallelScan</i>)
Trial 1	8530.572266	50.350719
Trial 2	8529.604492	50.367489
Trial 3	8529.864258	50.359295
Trial 4	8530.827148	50.359295
Trial 5	8554.600586	50.367489
Trial 6	8529.610352	50.360321
Trial 7	8529.788086	51.117058
Trial 8	8530.683594	50.372608
Trial 9	8529.041016	50.354847
Trial 10	8530.469727	50.370568
Mean (μ)	8532.506153	50.437968

Table 6: Raw Runtimes: Non-Supremum Simulation + Improved Reduction (ms)

Trial Index	GBM Kernel (<i>gbmKernel</i>)	Improved Reduction Engine
Trial 1	6389.247071	13.946886
Trial 2	6359.335449	13.939712
Trial 3	6358.755371	13.941766
Trial 4	6359.617188	13.940576
Trial 5	6358.805176	13.946816
Trial 6	6359.253418	13.944646
Trial 7	6359.627441	13.946368
Trial 8	6360.003418	13.944832
Trial 9	6386.854980	13.942784
Trial 10	6359.758789	13.944768
Mean (μ)	6365.125830	13.943914

References

- [1] Paolo Baldi. *Stochastic Calculus: An Introduction Through Theory and Exercises*. Universitext. Springer, Berlin, Germany, 2017. doi: 10.1007/978-3-319-62226-2.
- [2] M. Harris. How to implement performance metrics in cuda c/c++, 2012. <https://developer.nvidia.com/blog/how-implement-performance-metrics-cuda-cc/>, Last accessed on 2026-03-08.
- [3] D. Kirk and W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach (2nd ed)*. Morgan Kaufmann, 2012.
- [4] NVIDIA Corporation. *NVIDIA MathDx 25.12: Device Extensions for Mathematical Libraries*. NVIDIA Corporation, Santa Clara, CA, USA, 2025. URL <https://docs.nvidia.com/cuda/mathdx/index.html>. Version 25.12.
- [5] NVIDIA Corporation. *cuRAND Toolkit Documentation: Device API Overview*. NVIDIA Corporation, 2026. URL <https://docs.nvidia.com/cuda/curand/device-api-overview.html>. CUDA Toolkit Documentation.
- [6] NVIDIA Corporation. *cuRANDDx: Device-side Random Number Generation Extensions*. NVIDIA Corporation, Santa Clara, CA, USA, 2026. URL <https://docs.nvidia.com/cuda/mathdx/index.html>. Available within the NVIDIA MathDx and HPC SDK ecosystems.